

What Our Research Did — A Brief for My Supervisor

Privacy-Preserving, Fairness-Aware Federated Fraud Detection on Hyperledger Fabric

This document tells the story of the project in plain language. Technical terms are explained the first time they appear, and again in the glossary at the end. Every section says where the same material lives in the full thesis report. All numbers come from re-running our actual code.

1. The problem we started with

Banks want to catch fraud. The trouble is that any single bank only sees *its own* customers' transactions. A fraud pattern that is spread across many banks — the same stolen card tried at ten different institutions — is almost invisible to each bank on its own. If the banks could combine their data, their fraud detectors would get dramatically better.

But they can't simply pool the data. Privacy laws forbid sharing raw customer records; banks are commercial rivals who don't want to hand over their data; and there is no neutral, trusted party that everyone is willing to let hold the combined dataset. This is the deadlock our project tackles.

In plain terms: the deadlock Everyone would benefit from pooling data, but nobody is allowed to, wants to, or trusts anyone enough to actually do it. We need the *benefit* of pooling without the *act* of pooling.

There is a known idea that promises a way out, called **federated learning**.

In plain terms: federated learning (FL) Instead of sending your data to a central place, each bank trains a model on its own data, in-house, and only shares the trained model's *numbers* (its "weights") — never the raw transactions. A coordinator combines everyone's numbers into one shared, smarter model. The data never leaves the building; only lessons learned from it do.

Federated learning sounds like it solves everything. But the moment you try to actually run it between banks that distrust each other, four new problems appear that the textbook glosses over:

1. **Who runs the coordinator, and why trust them?** Whoever combines the models holds a position of power. Rivals won't accept one bank (or a third party) being in charge.
2. **Why contribute good data?** If everyone receives the same final model, a lazy bank can contribute garbage (or nothing) and still get the benefit — a "free-rider." Good contributors need to be *measured* and *paid* fairly.
3. **The shared numbers still leak.** It turns out a model's weights can, with effort, be reverse-engineered to reveal things about the data they were trained on. So even the numbers need protection.
4. **Someone might sabotage the shared model** on purpose by sending poisoned numbers.

The tempting pitch is that you can fix all four at once by bolting on four well-known tools — a *blockchain* for trust, *differential privacy* for leakage, *Shapley values* for fair payment, and *Krum* for sabotage-resistance (all explained below). The result, in theory, is a system that is private, fair, secure, and decentralised all at the same time.

Our research question is simply: does that pretty promise actually hold up when you build it for real? We took an existing, published fraud detector (the DB-BOA-ADTCN model of Prabanand & Thanabal, 2025) and moved it into this real federated, blockchain setting to find out.

↪ **Where this is in the report:** Chapter 1 (Introduction) sets up this problem and lists our objectives; Chapter 2 (Literature Review) explains where each of the four tools comes from and what gap we fill.

2. The four tools, in plain language

Before the story continues, here is what each bolt-on tool actually does. These four names recur throughout the report, so it is worth fixing them in mind.

In plain terms: blockchain / Hyperledger Fabric A shared digital ledger that no single party controls — every participant keeps a copy, and entries can't be secretly changed. It removes the need for a trusted middleman. **Hyperledger Fabric** is a specific, business-grade blockchain platform; we run a real one, not a pretend one. The small program that defines our rules on it (logging results, handing out reward tokens) is called a “chaincode.”

In plain terms: differential privacy (DP) A mathematical way to add carefully-calibrated random “noise” (small fudge factors) to the numbers a bank shares, so that no individual customer's data can be backed out of them — while the overall model is still useful. There is a dial, written ϵ (epsilon): *small ϵ = lots of noise = strong privacy but blurrier numbers; large ϵ = little noise = weak privacy but clearer numbers*. The whole trade-off in this thesis lives on this dial.

In plain terms: Shapley values A fair way, borrowed from game theory, to answer “how much did each player actually contribute to the team's success?” We use it to score how much each bank's data improved the shared model, so rewards can be split fairly. We represent each round's scores as a short list of numbers, φ (phi).

In plain terms: Krum A defence against sabotage. When combining everyone's model numbers, instead of naïvely averaging them (where one wildly poisoned set can drag the average off), Krum picks the one set of numbers that is most “in agreement” with the majority and throws out the outlier. It has a mathematical guarantee — but only when there are enough honest participants relative to attackers.

↪ **Where this is in the report:** Chapter 4 (Proposed Methodology) describes how we wired these four together into one pipeline, and Chapter 3 states our “threat model” (exactly which attacks we defend against).

3. What we built

We didn't just describe this on paper — we implemented the whole stack and made it run end to end:

- **The detector.** A fraud-classifier model (ADTCN) whose settings are tuned automatically by a search algorithm called **DB-BOA**, so a human doesn't have to hand-tune it. The same algorithm also picks which node “leads” each blockchain round.
- **The federation.** Three simulated banks — BankA, BankB, BankC — each train locally, and their models are combined through the pipeline: *add privacy noise (DP) → reject sabotage (Krum) → score contributions (Shapley)*.
- **The rewards.** Each bank's Shapley contribution score automatically determines its cut of a pool of reward “tokens,” recorded on the blockchain. Contribute more, earn more — no human judgement involved.
- **The blockchain.** A real Hyperledger Fabric network runs underneath, logging every round and keeping the token accounts.

But the *point* of the project was never just “assemble the stack.” The real question was the hard

one from Section 1: **when you actually combine privacy, fairness, and security, do they cooperate — or do they get in each other’s way?**

↔ **Where this is in the report:** Chapter 4 (Proposed Methodology) — the full pipeline, the DB-BOA tuning, and the chaincode.

4. The story of what happened (the heart of the thesis)

Here is the turn that makes this a piece of research rather than just an engineering demo.

When we turned on *strong* privacy — the privacy dial set to a tight $\epsilon = 1.0$ (lots of protective noise) — and ran the complete system honestly, it didn’t just get a bit worse. It **fell apart**. The combined model became useless: it stopped distinguishing fraud from non-fraud at all, effectively just labelling everything the same way.

In plain terms: why it fell apartThe privacy noise was being sprinkled over a *huge* set of numbers (the model’s $\sim 111,000$ weights). At strong privacy, that much noise drowned the real signal entirely. And Krum, the sabotage defence, couldn’t rescue it — by design Krum keeps just *one* bank’s noisy numbers and discards the rest, which throws away the averaging that would otherwise cancel some of the noise out.

A weaker project, faced with this, would have quietly turned the privacy dial down until the demo looked good again, and never mentioned it. We did the opposite. We decided this failure *was* the finding: **privacy and fairness are fundamentally in tension, and the interesting scientific question is to map exactly where, and why, and whether it can be fixed properly.**

So we asked: is privacy itself the problem, or is it *where we were applying the privacy noise*? We had been noising the giant pile of model weights and then trying to read each bank’s contribution through that fog. What if, instead, we left the model weights alone and applied the privacy noise *directly to the small contribution scores* (φ) — just three numbers instead of a hundred thousand?

That one change of *where* the noise goes is the key result of the thesis. Protecting three numbers needs far less noise than protecting a hundred thousand, so fair rewards survive at a much stronger privacy setting.

In plain terms: the fix, in one lineDon’t blur the whole model to hide a contribution — just blur the contribution score itself. Same privacy promise, a tiny fraction of the damage.

↔ **Where this is in the report:** Chapter 5 (Result Analysis), §6.2 (the collapse) and §6.6 (the privacy-vs-fairness characterisation and our fix). The reframed contribution is also stated in Chapter 1.

5. What we found — the actual numbers

Every figure here comes from re-running our code (saved in `final_report.data/`).

The detector works (and we report it honestly)

On the public ULB credit-card fraud dataset, our detector scored **99.85% accuracy**. But accuracy is misleading here, because only 0.17% of transactions are fraud — a lazy model that says “never fraud” already scores 99.83%. So we report **MCC** instead.

In plain terms: MCC (Matthews Correlation Coefficient)A single honest score from -1 to $+1$ that only looks good if the model gets *both* fraud and non-fraud right. 0 means “no better than guessing.” It can’t be fooled by the “always say normal” trick. Our detector scores **MCC 0.677** — genuinely useful.

Honest caveat we state openly: our *auto-tuned* detector does *not* beat a hand-set default — and it does not lose to one either. Measured across five seeds, the tuned configuration averages MCC 0.753 ± 0.055 against the default’s 0.706 ± 0.076 : a difference of $+0.047$ at $p = 0.34$, which is indistinguishable from zero. (An earlier draft of this brief reported the tuned detector *trailing* a default at MCC 0.785. That claim is withdrawn: a 13-run reproducibility study showed it was measuring CPU thread count rather than the optimiser.) So DB-BOA’s real value is *removing the manual tuning work*, not beating a human expert — and we say exactly that.

The centrepiece result: the privacy-vs-fairness fix

Moving the privacy noise from the model weights onto the small contribution scores keeps the reward split *honest* at privacy settings where the old approach pays people in the wrong order. We state this in the two forms that survive scrutiny; neither depends on an arbitrary cut-off, and both are measured in all three run conditions.

(1) The ordering. The contribution channel keeps rewards better-ordered than the weight channel at **essentially every privacy budget** — 11 of 11 on rank-correlation in all three conditions, and 11/11, 11/11 and 10/11 on the stricter mis-ranking-rate metric. The one counter-example sits at $\epsilon = 1$ on BankSim/entity-disjoint (weight mis-ranks 85/100, contribution 88/100), where *both* channels fail near-totally. This is a *paired* comparison — the same noise draw drives both channels in the same iteration — so more data changes its precision, not its direction. It has now held across three conditions, two sample sizes (100 and 1000 draws) and two metrics.

(2) Reward quality at a fixed, stated budget. At $\epsilon = 50$, the correlation between paid tokens and true contribution (−1 to 1):

Condition	Old way (noise on weights)	Our way (noise on scores)
ULB	−0.199 (rewards backwards!)	+0.995
BankSim, stratified	+0.017 (no signal)	+0.945
BankSim, entity-disjoint	−0.070 (no signal)	+0.805

That table is the centrepiece. At a genuinely private setting the old channel pays contributors in an order uncorrelated with — on ULB actively opposite to — what they actually contributed, and the new channel gets it nearly right. No threshold, no grid, no extrapolation.

Read before quoting: why we no longer headline a “60× more privacy” figure The factor is $\epsilon^*(\text{weight})/\epsilon^*(\text{score})$, where ϵ^* is the largest budget at which rewards are still mis-ranked *at all*. We extended the grid to $\epsilon = 30000$ and re-measured the deciding budgets at 1000 draws instead of 100. Three things came out of it. **(i) ϵ^* can only ever move up.** More draws, or more budgets, can only *find* additional mis-rankings — never remove one. Extending the grid raised BankSim/entity-disjoint from “ $\geq 10\times$ ” to $33\times$; the 1000-draw pass raised it again to $\geq 100\times$ and pushed it back outside our grid, so it is a lower bound once more, resting on a single mis-ranked draw in a thousand. Two increases from looking harder, in one day. **(ii) The factor is not stable under its own definition.** Setting the threshold at a small non-zero mis-ranking rate instead of “any at all” gives, for ULB / BankSim-stratified / BankSim-entity-disjoint: $60\times, 30\times, \geq 100\times$ at rate > 0 ; $20\times, 60\times, 33\times$ at rate > 0.01 ; and $33\times, 20\times, 33\times$ at rate > 0.05 . The value swings between $20\times$ and $\geq 100\times$ and the *ranking of the three conditions* changes with the choice. A number that mobile cannot carry a claim, so our summary is “**one to two orders of magnitude, definition-sensitive**”, with the ordering and the correlation table doing the real work. **(iii) What we did confirm:** thresholds that had rested on a single draw in 100 were **real, not flukes** — at 1000 draws they returned as 7, 11, 16 and 10 per 1000, all consistent with a true rate near 0.01. The earlier fragility was about precision, not about whether the effect existed.

Read before quoting: a reproducibility failure we are disclosing The ULB result we stored on 2026-08-30 does *not* reproduce under our current environment: 13 of 18 shared cells moved. Both BankSim conditions reproduce **bitwise**, and ULB reproduces *itself* on same-day re-run, so the pipeline is deterministic — but the cause of the drift is **unexplained**. We eliminated every code change on the path, all six configuration dicts, and the CPU thread count; the library version in use at the time is unrecoverable, because the version pin was added afterwards. Every sweep result now records its full software environment so this cannot recur. The ULB figures above come from re-verified 2026-09-04 runs and stand on their own evidence; they cannot be reconciled with the older stored number.

Security holds where it's supposed to — but security is not accuracy

We tested Krum against four kinds of sabotage attack, in the conditions where its mathematical guarantee is meant to apply (enough honest banks). Krum **caught and rejected the attacker in all 8 test cases**, keeping the shared model at $\approx 99.9\%$ balanced accuracy on ULB. Without Krum, the worst attack dragged a plain average down to $\approx 87.5\%$. So the security claim is real — within stated limits. **Catching the attacker is in fact our most robust result: 8 out of 8 in all three run conditions** (ULB, and BankSim under both a stratified and an entity-disjoint bank split).

Read before quoting: what Krum does *not* buy The accuracy Krum gains does **not carry over**. That +12.49 pp advantage over an unprotected average is a **ULB** number. Run the same experiment on BankSim and the advantage goes *negative* — Krum throws the attacker out and the model it picks is still worse than the plain average — in 7 of 8 cells on the stratified split and 8 of 8 on the entity-disjoint one. **Why it costs accuracy there is unexplained, and we say so rather than inventing a reason**. An earlier explanation — that it was caused by banks holding disjoint sets of customers — was withdrawn on 2026-09-04 by our own control experiment, which found the same cost on a split where the banks *share* customers. Two consequences for the writing: report “attacker rejected” and “accuracy gained” as **separate claims**, and retire the phrase “Krum’s value shows up under attack”.

Each ingredient’s effect, side by side (an honest ablation)

Setup	Accuracy	MCC	What it tells us
Plain averaging (FedAvg)	99.72%	0.569	decent baseline
+ Krum (anti-sabotage)	99.92%	0.776	best single setup
+ strong privacy ($\epsilon=1.0$)	99.83%	0.000	privacy alone collapses it
Full system at $\epsilon=1.0$	—	≈ 0	the collapse from Section 4

We *show* the collapse rather than hiding it — it is the evidence for our central point.

In plain terms: FedAvg The standard, naïve way to combine models in federated learning: just average everyone’s numbers together. Simple, but it has no defence against sabotage or privacy leakage on its own — which is why the other tools get added.

The blockchain numbers are measured, not invented

We measured our real Fabric network rather than guessing: each agreement round takes about **2.1 seconds** (a built-in timing floor of the platform), and it sustains about **40 transactions per second** before hitting a known congestion limit. (Earlier drafts had quoted made-up figures; these replace them.)

↪ **Where this is in the report:** Chapter 5 (Result Analysis) — §6.1 detector, §6.2 ablation, §6.6 the privacy-vs-fairness fix, the Byzantine-robustness section (Krum 8/8), and the Fabric-measurement section.

6. What we can honestly claim as our contribution

Three things, strongest first, in words that will survive a tough examiner:

1. **A new privacy-preserving reward mechanism** — the idea of putting the privacy noise on the *contribution score* instead of the *model weights*, which makes fair rewards survive rank-faithful rewards at a genuinely private budget (correlation +0.995 against −0.199 at $\epsilon = 50$ on ULB, same direction in all three conditions). The budget improvement is one to two orders of magnitude but sensitive to how the threshold is defined — see the caveat in §5; the new channel wins at *every* budget we tried, in all three). *This is the one genuinely new piece.*
2. **A clear characterisation of the privacy-vs-fairness trade-off** — real evidence that naïvely combining strong privacy with fair-reward scoring makes rewards *worse than random*, plus a map of exactly where that breakdown happens. *A legitimate “negative result” finding.*
3. **A complete, working, honestly-measured system** — all the pieces integrated and running on a real blockchain. *This is integration of known tools, and we label it as such.*

We are careful *not* to overclaim: we did not invent differential privacy, Krum, Shapley values, or Fabric, and we do *not* claim to be “first” to put fair rewards on a blockchain — an earlier system (FedCoin, 2020) did that, and we build on it.

In plain terms: why “negative results” still count as research Showing clearly and with evidence that a popular combination *doesn’t* work the way people assume — and explaining why — is a real scientific contribution. It saves the next team from the same trap and points to the right fix. Honesty here is a strength, not a weakness.

↪ **Where this is in the report:** Chapter 1 (contribution list) and Chapter 6 (Conclusion).

7. Where we fall short — our limitations

We state these plainly in the report. Examiners trust honest limitations far more than a flawless-looking story.

- **The “banks” are simulated, not genuinely separate institutions.** We didn’t have several real banks’ data, so we split public datasets into portions. This has since been strengthened — a second dataset (BankSim, 594,643 transactions with 4,112 real customer IDs) was added and the system experiments re-run on it under *both* a volume split and a genuinely customer-disjoint one, so five of our six headline claims now rest on two datasets rather than one. But even the customer-disjoint split deals out the customers of a *single* source, so we still never test banks whose data comes from genuinely *different* institutions.
- **The fairness/reward defence only works in some situations.** It can isolate a coordinated *majority* of cheaters, but it does *not* catch a lone attacker or a quiet free-rider (who contributes nothing but isn’t actively malicious). Fixing free-riding needs an extra rule we leave for future work.
- **“Secure” and “Scalable” are bounded claims, not blanket ones.** Krum’s guarantee only holds when honest participants sufficiently outnumber attackers; a very subtle attacker that stays close to honest behaviour could still slip through. And “scalable” refers to our *contribution-scoring* method, not the blockchain’s raw speed (which has a clear ceiling).
- **Some results are from a simulation, not a live deployment.** The blockchain speed

numbers and the leader-picking behaviour come from a single-computer test network, not a real multi-bank, multi-city deployment. The *patterns* transfer; the exact numbers wouldn't.

- **A small measurement caveat on the contribution scores.** They're computed on held-out data drawn from the same pool as the training data, so there's a minor, disclosed circularity — not an independent bank's data.

↔ **Where this is in the report:** Chapter 5 / Chapter 6 — the “Threat model” paragraph and the “Limitations and disclosures” block (drafted in `final_report_data/REWRITE_08_limitations_disclosures.md`).

8. If my supervisor reads only one paragraph

*We built a system that lets rival banks jointly train a fraud detector without sharing raw data, on a real blockchain, with privacy protection, fair contribution-based rewards, and sabotage resistance. Our central finding is that these good properties **fight each other**: turn privacy up to a serious level and the fair-reward mechanism breaks so badly it pays the worst contributors the most, and the model collapses. We pinned down exactly why — the privacy noise was being applied in the wrong place — and fixed it by noising the small contribution score instead of the huge model, which restores fair rewards at a genuinely private budget (correlation +0.995 against -0.199 at $\epsilon = 50$ on ULB). The corresponding budget-improvement factor spans one to two orders of magnitude and is definition-sensitive ($20\times$ to $\geq 100\times$, see §5), so we report the ordering rather than a single multiplier; what holds everywhere is that the new channel wins at every privacy budget we tried). The sabotage defence catches the attacker every time in all three conditions — though on our second dataset catching it costs accuracy instead of gaining it, which we report as measured and do not explain away — and our blockchain numbers are measured rather than invented. So the contribution is: one new mechanism, one honest characterisation of a real trade-off, and a complete working system — with its limitations (simulated rather than genuinely multi-institution federation, conditional reward defence, simulated network) stated openly.*

9. The whole story in one sitting (a plain walkthrough)

If the earlier sections still feel like a pile of separate parts, here is the same research told as a single, continuous story — no jargon assumed.

The setting

Imagine several banks that each want to catch fraudulent transactions. A better fraud detector needs more data — but **no bank will hand its customers' records to the others** (privacy law, and they are rivals). The standard way out is **federated learning**: each bank trains a model on its *own* data and shares only the trained model, never the raw data. A coordinator combines these into one stronger shared model. Everyone benefits; nobody exposes raw data.

Three things you need at once — and they fight

A real federated system between distrustful banks has to do three jobs at the same time, and they pull against each other:

1. **Fair pay (incentive).** One bank has excellent data, another has almost none. Pay them equally and the good one walks away. So you must *measure* how much each bank actually helped (**Shapley value**) and pay it accordingly, recorded tamper-proof on a **blockchain**.
2. **Privacy.** Even sharing the *model* can leak information about individuals. The defence is **differential privacy**: add random *noise*. More noise = more privacy.

3. **Security.** A malicious bank might send a poisoned model to wreck the shared one. The defence is **Krum**, which discards the suspicious outlier instead of blindly averaging.

The conflict at the heart of the thesis

Here is the tension everything hangs on: **privacy sabotages fairness.** To get privacy you add noise — but that same noise scrambles the contribution measurement (Shapley). When the measurement is scrambled, the blockchain **pays the wrong banks**: the free-rider gets rewarded, the hard worker gets shortchanged. The incentive becomes a lie. So the honest research question becomes: “*When does this reward system actually stay honest, instead of paying the wrong people?*”

In plain terms: why this is the novelty All the parts already exist (federated learning, Shapley, privacy, Krum, blockchain). What is new is *mapping out exactly when the combination breaks — and fixing it*. That is a characterisation contribution, not a new-algorithm claim.

What we discovered (the negative result)

We measured it. Adding the privacy noise the “obvious” way — onto the **model weights** ($\sim 100,000$ numbers) — the rewards only stay correct when privacy is *almost switched off* ($\epsilon \approx 3000$). In plain terms: with the obvious design you can have correct rewards *or* real privacy, but not both. That is a real, useful finding on its own.

What we built (the fix — the centrepiece)

Then the key insight: the noise does so much damage because it is dumped onto 100,000 numbers. But you do not need to privatise the weights to protect the reward calculation — you only need to privatise the **contribution scores themselves**, which are just a handful of numbers (one per bank). So we flip the order: compute the honest scores on the *clean* models first, *then* add the privacy noise to those few numbers before publishing them. Same privacy guarantee, but noising a handful of numbers instead of 100,000 — so the damage drops by roughly two orders of magnitude. Honest rewards now survive at a genuinely private setting ($\epsilon \approx 50$).

In plain terms: the punchline *Where* you inject the privacy noise relocates the honesty boundary by one to two orders of magnitude — we never found the point where the old channel stops failing, so that number is a floor. We turned a “you can’t have both” negative result into a “here’s how you get both” positive one.

Why the other title words are earned

Secure — Krum catches poisoning that plain averaging misses. **Scalable** — the exact Shapley sum is too expensive, so we use a sampling shortcut that scales, plus we *measured* real blockchain throughput. **Consensus / RL** — the blockchain integration, with a learning agent choosing which node leads each round. These are honest integration and measurement, not new inventions.

↔ **Where this is in the report:** This walkthrough condenses Chapters 1, 4 and 5; the fix and its numbers are §6.6.

10. Glossary (quick reference)

Federated learning (FL)

Training a shared model from many parties’ data without any of them sending their raw data — only model numbers are shared.

Hyperledger Fabric

A real, business-grade blockchain platform; we run one as the trusted, no-middleman coordinator. The on-chain rule program is the “chaincode.”

Differential privacy (DP) / ϵ

Adding tuned random noise to shared numbers so no individual can be identified. ϵ is the dial: small = strong privacy + blurry; large = weak privacy + clear.

Shapley value (φ)

A fair score of how much each participant contributed to the team result; we use it to split rewards.

Krum

A defence that picks the most-agreed-with model and discards sabotage outliers, with a guarantee when honest participants are numerous enough.

FedAvg

The plain baseline: just average everyone’s model numbers, with no built-in defences.

MCC

An honest accuracy score (−1 to +1) that isn’t fooled when fraud is rare; 0 = guessing.

ADTCN / DB-BOA

The fraud-detector model, and the search algorithm that auto-tunes it (and picks blockchain leaders), respectively.

Byzantine attack

Jargon for a participant deliberately sending bad/poisoned data to wreck the shared model.

Free-rider

A participant who takes the shared model’s benefit while contributing little or nothing useful.

Full report chapter map: Ch1 Introduction · Ch2 Literature Review · Ch3 Requirements, Impacts & Constraints · Ch4 Proposed Methodology · Ch5 Result Analysis · Ch6 Conclusion. Verified numbers: `final_report_data/`. Anticipated defence questions: `defense_questions.md`.